From Optimal Control to Diffusion Policy

A Trajectory-Optimization → TVLQR → Behavioural-Cloning Pipeline for Double-Pendulum Swing-Up

Group Project TN2 — Technical Documentation

June 26, 2026

Contents

1 Introduction

The goal of this project is to compare the performance of 2 different Imitation Learnig control methods. The first method is a behavioural cloning approach, where we use a dataset of expert demonstrations to train a policy that mimics the expert's actions. The second is a diffusion policy approach, where we use a diffusion model to learn a policy that generates actions from noise based on the current state of the system. We will evaluate the performance of both methods in the **double pendulum swingup task** and the **stacking problem**.

2 Behavioural Cloning - Double Pendulum Case

2.1 Behavioural Cloning model and policy training

We trained a neural network with a horizon of 7 actions ahead of the current one. Therefore the policy the agent attempts to learn is:

$$\pi = (a_{t:t+7}|s_t) \tag{1}$$

To train our model we first preproccess the expert data and instead of the state s_t we input the feature transformation $\phi(s_t)$. Additionally, we separated the expert data into training and validation data. The training data comprise 90% of the expert data while testing data comprise 10% of the expert data. We have performed normalisation in both the actions a_t and the velocities $\dot{q}_t$ of the pendulum. In order to train this model we did not utilize test data since we test it in the simulation environment.

The architecture of the NN is depicted in Figure 1

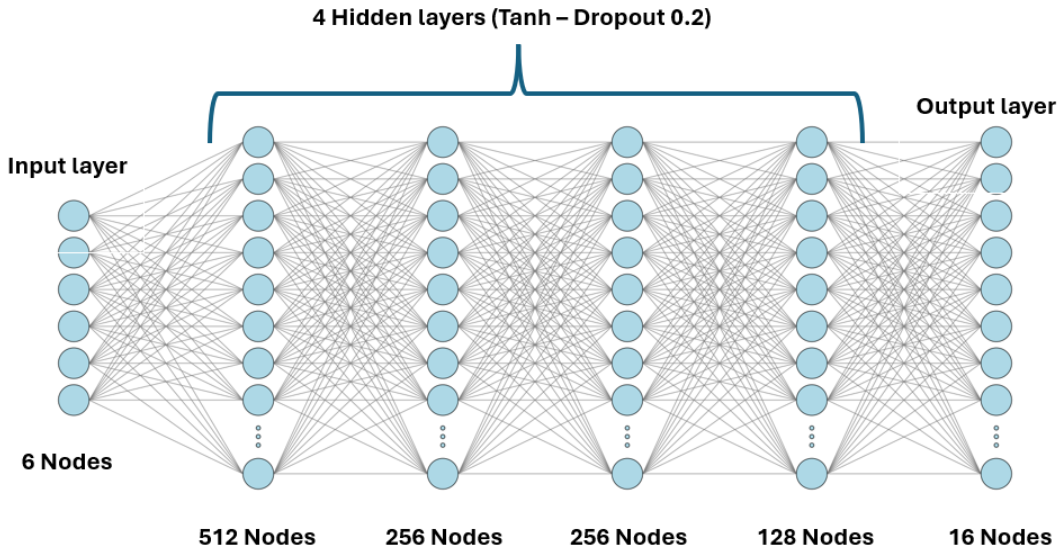


Figure 1: Architecture of BC policy model for the double pendulum example.

As it shown above the network has an input layer of size 6 and 4 hidden fully connected layers while the output of the output layer is 16 which is then reshaped to represent a set of 8 consecutive actions in the form of a 8×2 matrix. The activation function of each layer is the hyperbolic tangent ($tanh(x)$). Additionally, in the output of each hidden layer we have added a dropout layer with a dropout rate of 20% which is only active during training. Regarding the training hyperparameters are depicted in Table 1. In Figure 2 the training and test losses are depicted with respect to the training epochs. The training finishes early so as to not overfit to the expert trajectories.

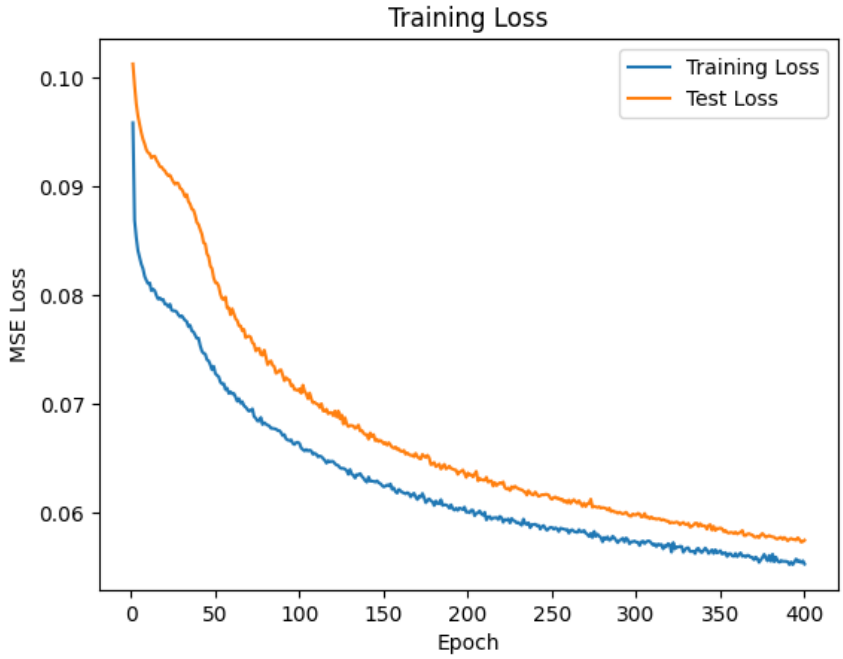


Figure 2: Train and Test loss curves of the BC policy model.

Table 1: Behavioural Cloning policy model and training hyperparameters (defaults).

Parameter	**Symbol**	**Value**
Feature dimension	—	6 per state
Action prediction horizon	H	8
Optimizer / LR	—	Adam, 10^{-3} (cosine)
Batch size / epochs	—	512 / 400
Loss Function	—	MSE

2.2 Diffusion

In order to understand the diffusion policy approach, we first need to understand the concept of diffusion models. Diffusion models are a class of generative models that learn to generate structured data (in our case, action sequences) by iteratively denoising a sample of noise. The formulation we summarise below follows the treatment in *Understanding Deep Learning* [3].

Parameter Setup

For the diffusion model, we first need to define the time parameters of the noising and denoising process.

- **Time Horizon** T is the number of time steps in the diffusion process.
- β **Schedule** β_t (with t $\in \{1, 2, \ldots, T\}$) is a set of hyperparameters that control how quickly the noise is added to the data during the forward diffusion process. It is a monotonic increasing sequence of values that determine the variance of the noise
- **Noise Schedule** $\alpha_t = 1 - \beta_t$ is a set of hyperparameters that control how quickly the noise is removed.
- **Cumulative Noise Schedule** $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$

So we can define the forward diffusion process as follows:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \tag{2}$$

where x_0 is the original data (action sequence) and x_t is the noisy version of the data at time step t. So now we need to define a procedure to undo the noise in T steps. The reverse diffusion process is defined as follows:

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \cdot x_t - \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} \cdot \epsilon(x_t, t), \quad \epsilon(x_t, t) \sim \mathcal{N}(0, I) \tag{3}$$

However, we do not have access to the true noise ϵ that was added during the forward diffusion process. Instead, we train a neural network $\epsilon_\theta(x_t, t)$ to predict the noise given the noisy data x_t and the time step t.

Algorithm 1 Diffusion Model Training

Require: Training dataset $\mathcal{D}$, diffusion steps T

$\theta \leftarrow$ init ▷ initialize network parameters
repeat
  for all $x_0 \in \mathcal{D}$ **do**
    $t \sim \text{Uniform}\{1, \ldots, T-1\}$ ▷ sample a diffusion timestep
    $\epsilon \sim \mathcal{N}(0, I)$ ▷ sample Gaussian noise
    $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$ ▷ forward (noising) step
    $\hat{\epsilon} = \epsilon_\theta(x_t, t, c)$ ▷ network predicts the noise
    $\mathcal{L} = \|\hat{\epsilon} - \epsilon\|_2^2$ ▷ ϵ-prediction MSE loss
  end for
  $\theta \leftarrow \theta - \eta\, \nabla_\theta \bar{\mathcal{L}}$ ▷ update on the average batch loss
until convergence

Algorithm 2 Reverse Diffusion Sampling (deterministic, control-time)

Require: Trained network ϵ_θ, conditioning vector c

$x \sim \mathcal{N}(0, I)$ ▷ start from the pure-noise prior
for $t = T-1, T-2, \ldots, 1$ **do**
  $\hat{\epsilon} = \epsilon_\theta(x, t, c)$ ▷ predict noise
  $x \leftarrow \frac{1}{\sqrt{\alpha_t}} \left(x - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\, \hat{\epsilon} \right)$ ▷ posterior-mean (denoising) step
  $x \leftarrow x + \sigma_t\, \epsilon, \quad \sigma_t = \sqrt{\beta_t}$ ▷ ancestral noise — **omitted** (deterministic)
end for
return x ▷ denoised action chunk $\tilde{a}^{1:H}$

Note that at control time we use the *deterministic* variant shown in Algorithm 2: the ancestral-noise term $\sigma_t \epsilon$ (greyed out above) is *not* added, so each reverse step returns the posterior mean. Dropping it sharply lowers the action variance, which is essential when stabilizing the unstable upright equilibrium.

2.3 Generating Dataset

Both learning methods are trained by imitation, so they are only ever as good as the expert demonstrations they are shown. We therefore generate the dataset with a classical optimal-control pipeline: we (i) define a physics model of the double pendulum, (ii) solve a trajectory-optimization problem for a nominal swing-up, (iii) wrap that nominal in a time-varying LQR (TVLQR) feedback controller, and (iv) roll the controller out *inside the simulator* to harvest dynamically consistent

state–action pairs. The result is a set of expert trajectories that both the behavioural-cloning and diffusion-policy models clone.

2.3.1 Dynamics Model

The state is $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]^\top$, where q_1 is the shoulder and q_2 the elbow angle, and the control $u = [u_1, u_2]^\top$ collects the two joint torques. We use $q_1 = 0$ for the stable hanging-down configuration and $q_1 = \pi$ for the upright goal. The planar two-link equations of motion are

$$M(q)\,\ddot{q} = u + G(q) - C(q,\dot{q})\,\dot{q} - b\,\dot{q} - f\,\tanh(\dot{q}/\epsilon)\,, \tag{4}$$

with mass matrix $M(q)$, Coriolis/centrifugal matrix $C(q,\dot{q})$, gravity vector $G(q)$, viscous damping b and Coulomb friction f. The non-smooth dry friction $\operatorname{sign}(\dot{q})$ is replaced by a smooth $\tanh(\dot{q}/\epsilon)$ ($\epsilon = 0.05$ rad/s) so the term is differentiable for the gradient-based solver. Crucially, every physical parameter (masses, lengths, link inertias, damping and friction) is read *directly* from the MuJoCo model file `dp.xml`, so the planner and the simulator used for deployment describe the exact same system and cannot silently drift apart.

2.3.2 Trajectory Optimization

The nominal swing-up is found by direct trapezoidal collocation, transcribed into a nonlinear program (NLP) and solved with IPOPT. Discretizing the horizon into $N+1$ knots, the decision vector stacks every state and control, $z = [x_0, \dots, x_N, u_0, \dots, u_N]$, and we minimize the quadratic cost

$$J(z) = \tfrac{1}{2}\sum_{k=0}^{N-1}\Big[(x_k - x_g)^\top Q\,(x_k - x_g) + u_k^\top R\,u_k\Big] + (x_N - x_g)^\top Q_f\,(x_N - x_g), \tag{5}$$

subject to the trapezoidal collocation (dynamics) defects, the boundary conditions, and the actuator torque limits:

$$x_{k+1} - x_k - \tfrac{\Delta t}{2}\big(f(x_k,u_k) + f(x_{k+1},u_{k+1})\big) = 0, \quad k = 0,\dots,N-1, \tag{6}$$

$$x_0 = x_{\text{init}}, \quad x_N = x_g = [\pi,0,0,0]^\top, \tag{7}$$

$$|u_k| \le \tau_{\max}. \tag{8}$$

The objective gradient and the constraint Jacobian are obtained by automatic differentiation (JAX), and the problem is compiled once and reused across solves. The weighting matrices Q, R trade off tracking accuracy against control effort; sweeping them, together with the peak-torque limit and the initial condition, produces a diverse family of nominal swing-ups rather than a single trajectory.

2.3.3 TVLQR Tracking and Closed-Loop Rollout

A nominal trajectory (x^*, u^*) on its own is open-loop and brittle: applied verbatim, the smallest model mismatch or numerical drift sends this chaotic system off course. We therefore linearize the dynamics about the nominal and compute time-varying LQR feedback gains K_t, yielding the tracking law

$$u_t = u_t^* - K_t\,(x_t - x_t^*). \tag{9}$$

A nearest-neighbour rule re-locks the controller onto the closest reference state (and blends the nearby gains) whenever the deviation exceeds a threshold, giving it a measure of recovery. This controller is rolled out inside the *actual* MuJoCo simulator, and at every control step (control period $\Delta t = 0.05$ s, i.e. 20 Hz) we log the observed state together with the commanded torque. Because each sample is produced by stepping the same simulator on which the policy is later evaluated, the dataset is dynamically valid by construction; cloning it cannot produce a policy that is "correct" for a physics model it never runs on.

2.3.4 DAgger-Style Coverage

A policy trained only on the nominal states suffers from compounding error: its own small prediction errors push it into off-nominal states it was never shown, and the errors snowball. To cover this recovery distribution, during the swing-up phase we perturb the *applied* torque with Gaussian noise ($\sigma = 0.15\,\tau_{\max}$) while still logging the *clean* commanded torque as the supervised target. Each sample then encodes "from this (possibly off-nominal) state, the expert would command this torque"—a DAgger-style relabelling that teaches recovery. The noise latches off once the system first reaches upright, leaving the fragile balancing phase undisturbed so the rollout still succeeds.

2.3.5 Dataset Composition

The data is gathered in two regimes:

- **Swing-up rollouts** start near hanging-down (angles in ± 0.6 rad, velocities in ± 1.0 rad/s) and run for up to 200 control steps. One in four is run noise-free to retain clean nominal behaviour.
- **Upright-hold rollouts** start *perturbed* about the upright equilibrium ($q_1 = \pi \pm 0.25$ rad, etc.) and are short and noise-free. Their purpose is to record the stabilizing deviation→corrective-torque map, without which the policy only ever sees the exact fixed point and never learns to balance.

Only rollouts that actually reach and hold the upright are kept. This yields approximately 600 trajectories—predominantly swing-up, with the remainder holding. Each trajectory is stored as an HDF5 group containing `states` $\in \mathbb{R}^{T\times 4}$ and `actions` $\in \mathbb{R}^{T\times 2}$ in `results/expert_trajectories.h5`, the common format consumed by both the behavioural-cloning and diffusion-policy training scripts.

Table 2: Key dataset-generation parameters.

Parameter	**Symbol**	**Value**
State / action dimension	n_x, n_u	4, 2
Upright goal state	x_g	$[\pi, 0, 0, 0]^\top$
Control period (rate)	Δt	0.05 s (20 Hz)
Friction smoothing	ϵ	0.05 rad/s
Swing-up exploration noise	σ	$0.15\,\tau_{\max}$
Swing-up rollout length	—	200 steps
Approx. trajectories kept	—	$\sim$600

2.4 Diffusion Policy Model

We follow the *Diffusion Policy* formulation of Chi et al. (2023): instead of regressing a single action from the current state, the network models the conditional distribution of a short *action chunk* given a window of recent observations, and samples from it by iterative denoising. The forward and reverse processes are exactly those of Section 2.2 above; what follows is the concrete realization for the double pendulum, with particular attention to the input representation, the normalization, and the handful of engineering choices that were necessary before the policy could actually swing up *and* balance.

2.4.1 Conditioning and Action Representation

The diffusion happens over an action chunk $a^{1:H}$ of $H = 10$ future torque pairs; the network is *conditioned* on a context vector c that is left clean (never noised). The context is built from the last $K = 4$ observed states plus the fixed goal. Two representation choices proved important:

Angular features. Feeding raw angles is harmful because q wraps at $\pm\pi$: two physically identical configurations can be numerically far apart, and the discontinuity sits right where the swing-up passes. We therefore map every state through a wrap-free angular feature transform

$$\phi(x) = \big[\sin q_1,\ \cos q_1,\ \sin q_2,\ \cos q_2,\ \tilde{\dot{q}}_1,\ \tilde{\dot{q}}_2\big] \in \mathbb{R}^6. \tag{10}$$

Min–max normalization. Velocities and torques live on very different scales from the unit-circle angle features, so both are min–max scaled into $[-1, 1]$,

$$\tilde{\dot{q}} = 2\,\frac{\dot{q} - \dot{q}_{\min}}{\dot{q}_{\max} - \dot{q}_{\min}} - 1, \qquad \tilde{a} = 2\,\frac{a - a_{\min}}{a_{\max} - a_{\min}} - 1, \tag{11}$$

with the per-dimension $(\min, \max)$ statistics computed once and *saved*, so the network output can be de-normalized back to physical torques at deployment. The diffusion target is the normalized action chunk $\tilde{a}^{1:H}$.

The conditioning vector concatenates the K feature-mapped history states with the goal features,

$$c = \big[\phi(x_{i-K+1}),\ \ldots,\ \phi(x_i),\ \phi(x_g)\big] \in \mathbb{R}^{(K+1)\cdot 6} = \mathbb{R}^{30}. \tag{12}$$

Including the velocity-carrying history (rather than a single state) gives the policy the information it needs to act on this second-order system. The goal token leaves the architecture *able*, in principle, to be retargeted to other set-points, but every trajectory in our dataset shares the single upright goal $x_g = [\pi, 0, 0, 0]^\top$; the goal input is therefore effectively constant, and the network never actually learns to condition on it. Genuine retargeting to a different goal would require demonstrations at that goal. Windows are edge-padded at the trajectory ends by repeating the first state / last action.

2.4.2 Noise-Prediction Network

The network $\epsilon_\theta(\tilde{a}_t^{1:H}, t, c)$ predicts the noise added to the action chunk and is trained with the standard ϵ-MSE objective. We implement two interchangeable backbones behind a common `forward`(a_{noisy}, t, c) contract:

- **MLP** — flattens the chunk to $H \cdot n_u$, concatenates a learned embedding of the (normalized) diffusion timestep and of c, and passes it through a SiLU MLP. It ignores the temporal structure of the chunk.
- **Trajectory Transformer** (default) — treats the H actions as a token sequence with learned positional embeddings, and *prepends two context tokens* (one for the diffusion timestep, one

for c) so every action token can attend to them. It uses pre-norm encoder layers with GELU activations; only the H action positions are projected back to torques. This preserves the temporal correlation within a chunk that the MLP discards.

The diffusion timestep enters as a sinusoidal embedding (transformer) or a small learned embedding (MLP), with t normalized to $[0, 1]$ by dividing by T.

2.4.3 Noise Schedule and Terminal SNR

We use a linear β schedule, $\beta_t \in [3 \times 10^{-4}, 0.1]$, over the $T = 20$ diffusion steps. Sampling starts from pure Gaussian noise, so the inference prior only matches the forward marginal $q(x_T \mid x_0)$ to the extent that the terminal signal-to-noise ratio is small — ideally the zero-terminal-SNR condition $\bar{\alpha}_T \approx 0$ (Lin et al. 2023). The upper end of the schedule is therefore deliberately large: an earlier, milder `end_beta` left far more of the action signal intact at x_T, widening the train/inference mismatch, and raising it reduces that residual.

How far this goes depends on *both* `end_beta` and the number of steps, since $\bar{\alpha}_T = \prod_{t=1}^{T}(1 - \beta_t)$ keeps shrinking as more steps accumulate noise. At the short horizons we sample with, the condition is approached but not reached: at $T = 20$ this schedule gives $\bar{\alpha}_T \approx 0.35$, so roughly 60% of the signal amplitude ($\sqrt{\bar{\alpha}_T}$) is still present at x_T, and only at much larger T (e.g. $T \approx 100$, where $\bar{\alpha}_T \approx 0.006$) is the terminal SNR effectively zero. We therefore do *not* claim a true zero-terminal-SNR prior; the enlarged `end_beta` shrinks the prior mismatch enough to denoise reliably at the low, fast-to-sample T that the closed-loop controller depends on.

2.4.4 Training Tricks

- **Trajectory-level split, train-only stats.** The data is split 70/15/15 into train/val/test *by trajectory* before any window slicing, so no two windows of the same trajectory straddle the boundary. Normalization statistics are computed on the *train fold only* and reused for val/test; letting the held-out folds normalize themselves would leak their distribution into the inputs.
- **EMA weights.** We keep an exponential moving average (decay = 0.999) of the network weights and use the EMA copy for all inference. The EMA is a much smoother estimate of the learned map; near the *unstable* upright equilibrium this matters concretely, because a jittery gain perturbs the torque and topples the pendulum. The EMA model is what gets saved to the checkpoint.
- **Optimizer / schedule.** Adam with a per-step cosine-annealing learning-rate schedule, plus mixed-precision (AMP) training on GPU.

2.4.5 Inference and Control Tricks

At control time the policy denoises an action chunk via DDPM ancestral sampling (reverse diffusion over the $T - 1$ denoising steps), conditioned on the current history and goal, then de-normalizes the result back to physical torques. Two choices are what make the closed loop stable rather than merely plausible:

- **Deterministic (noise-free) sampling for control.** The ancestral noise injected at each reverse step is dropped, so the sampler returns the posterior mean. This sharply lowers action variance — essential when *stabilizing* the unstable upright, where stray sampling noise on the torque is enough to drop the pendulum. (Stochastic sampling is kept available for when sample diversity, rather than control, is the goal.)
- **Short execution horizon.** Although the network predicts $H = 10$ actions, only the leading n_{exec} are executed before re-planning (receding-horizon action chunking). In practice executing a single action per re-plan is what reliably swings up; committing to longer open-loop chunks lets the chaotic dynamics diverge from the plan.

Table 3: Diffusion-policy model and training hyperparameters (defaults).

Parameter	**Symbol**	**Value**
Observation history	K	4 states
Feature dimension	—	6 per state
Action prediction horizon	H	10
Conditioning dimension	—	30
Diffusion steps	T	20
β schedule	β_t	linear $[3\times10^{-4}, 0.1]$
EMA decay	—	0.999
Optimizer / LR	—	Adam, 10^{-4} (cosine)
Batch size / epochs	—	256 / 400
Default backbone	—	Transformer (d=128, 4 heads, 2 layers)

2.5 Comparison

We evaluate both policies on the same double-pendulum swing-up task with the shared `evaluate_methods.py` harness, reporting success rate (nominal and over $n = 100$ random initial conditions with observation noise $\sigma = 0.025$), time-to-success, trajectory quality, stabilization accuracy, control smoothness, per-step inference cost, and robustness to increasing observation noise.

2.5.1 Aggregate metrics

Figure 3 collects the headline metrics. Both methods are essentially perfect at the task itself: behavioural cloning (BC) succeeds on all nominal and all 100 random-init trials (100%), and the diffusion policy matches it on the nominal case while losing ten random-init trials ($90/100 = 90\%$). The differences therefore live not in *whether* the pendulum is swung up but in *how*. The diffusion policy reaches the upright noticeably faster (5.07 s vs. 6.33 s) and with substantially higher and more consistent trajectory quality: its integrated absolute angle error (IAE) is markedly lower than BC's (8.98 vs. 14.6 rad·s, a $\sim$40% reduction) and its trial-to-trial spread is far tighter (std 3.2 vs. 7.7 rad·s), whereas BC's large error bar shows it occasionally takes long, meandering swing-ups.

The trade-off appears once the pendulum is upright. The two settle to essentially the same steady-state angle error (0.074 rad for BC vs. 0.071 rad for the diffusion policy), but BC holds that equilibrium far more smoothly: its mean torque increment (0.0033 Nm/step) is roughly 8$\times$ smaller than the diffusion policy's (0.0255 Nm/step), and its positional jitter at the goal is about an order of magnitude lower. Finally, the methods differ by more than an order of magnitude in compute: a single BC forward pass costs $\sim$0.27 ms, while one diffusion control step — which requires the full iterative denoising chain — costs $\sim$7.6 ms (note the logarithmic axis).

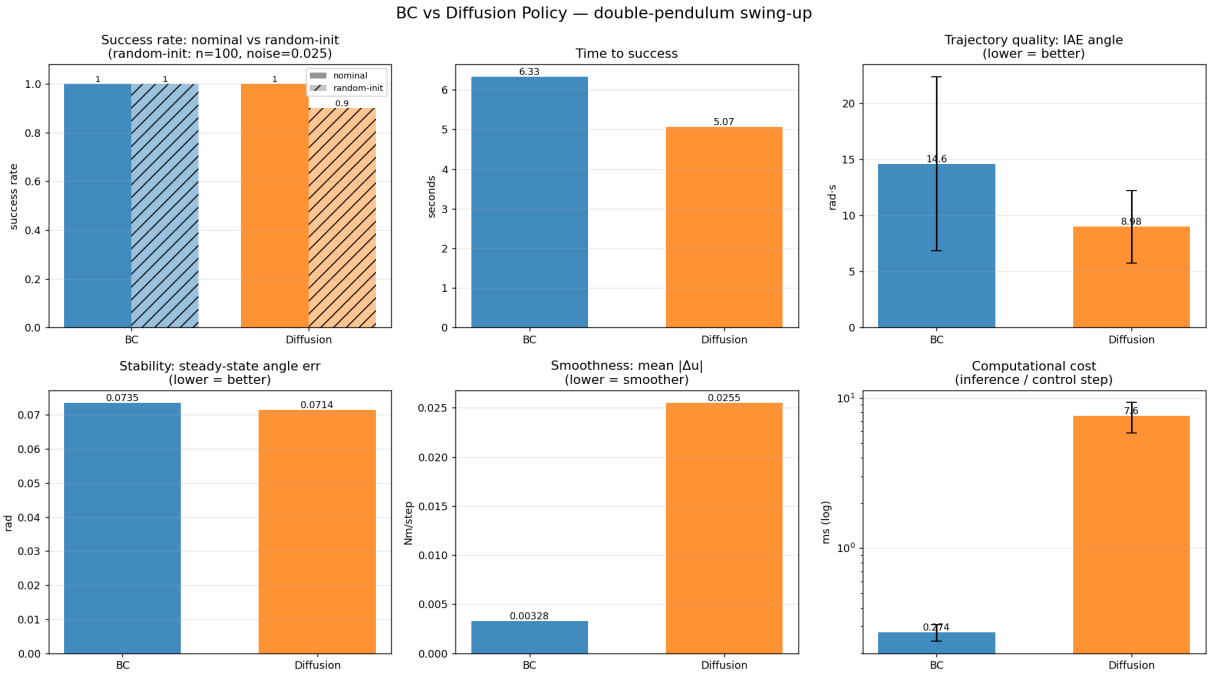


Figure 3: Aggregate metrics: BC vs. diffusion policy.

2.5.2 Robustness to observation noise

Figure 4 sweeps observation noise (Gaussian, σ on angles and 5σ on velocities) and reports the success rate at each level. At low noise both are near-perfect, with BC holding 100% all the way through $\sigma = 0.03$ while the diffusion policy already slips slightly (95% at $\sigma = 0.02$, 75% at $\sigma = 0.03$). Beyond that both degrade, but BC degrades more gracefully: at $\sigma = 0.04$ BC still holds 45% against the diffusion policy's 15%. At the most severe level ($\sigma = 0.05$) both collapse to 0%. The diffusion policy's heavier reliance on a short observation *history* (the $K = 4$ state window) plausibly makes it more sensitive to corrupted inputs than BC's single-state map.

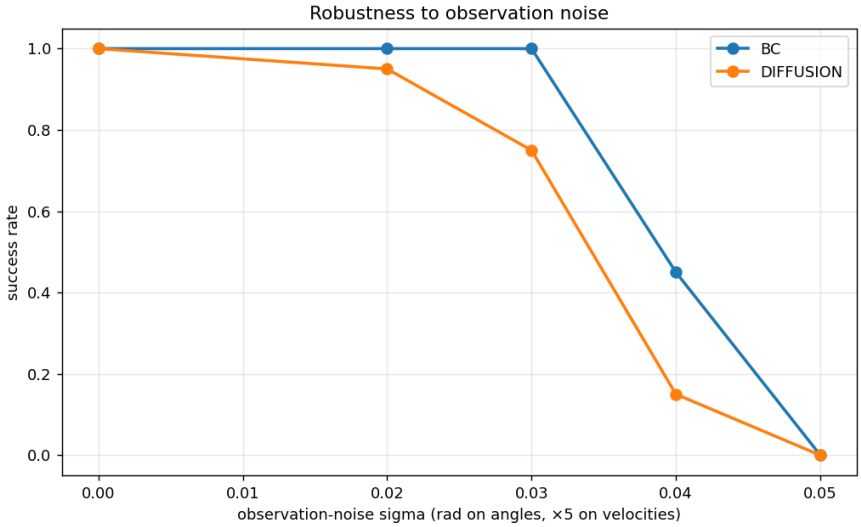


Figure 4: Robustness to observation noise.

2.5.3 Example trajectories

Figures 5 and 6 show a representative (best nominal) trial and make the smoothness/accuracy trade-off concrete. In the state plot both policies drive all four states to the upright goal, with the diffusion policy exhibiting a slightly faster, sharper rise. The control plot is the clearest discriminator: BC's torque settles to a near-constant, almost-zero hold once upright, whereas the diffusion policy's torque *chatters* continuously at high frequency throughout the hold phase. This persistent micro-correction is exactly what the high mean-$|\Delta u|$ in Figure 3 measures, and it stems from re-sampling a fresh action chunk every control step near the unstable equilibrium — even with deterministic sampling, tiny variations in the denoised output translate into a jittery torque.

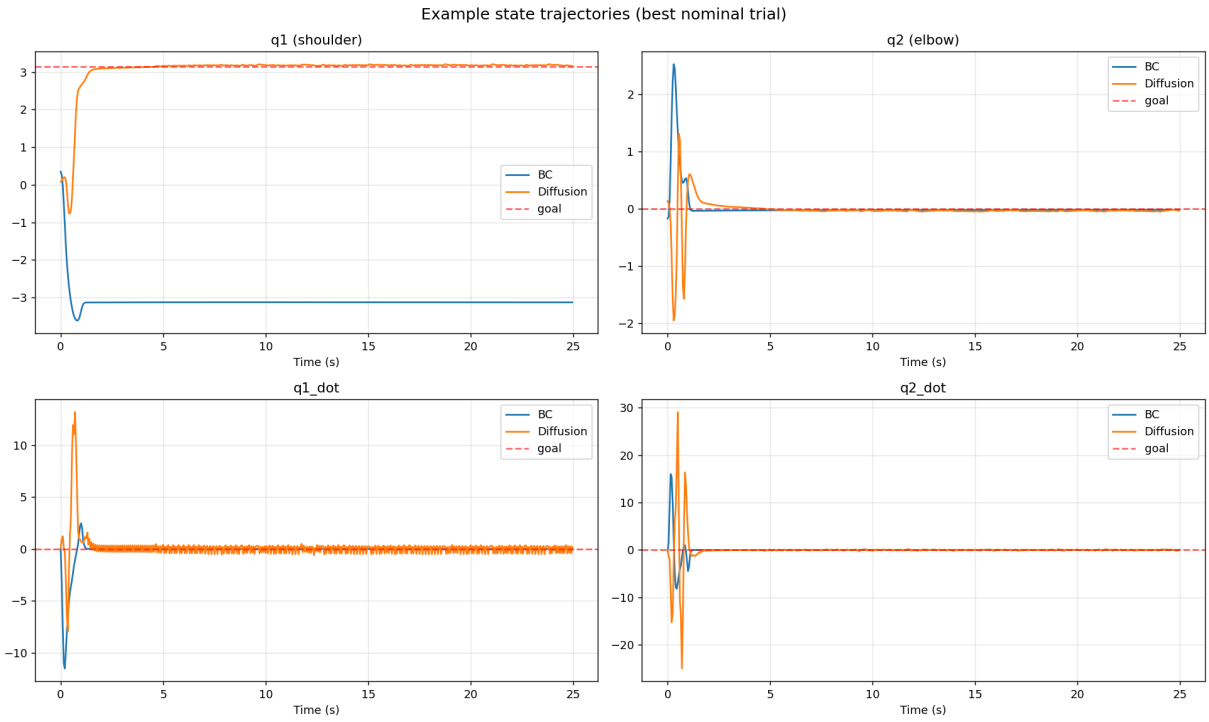


Figure 5: Example state trajectories.

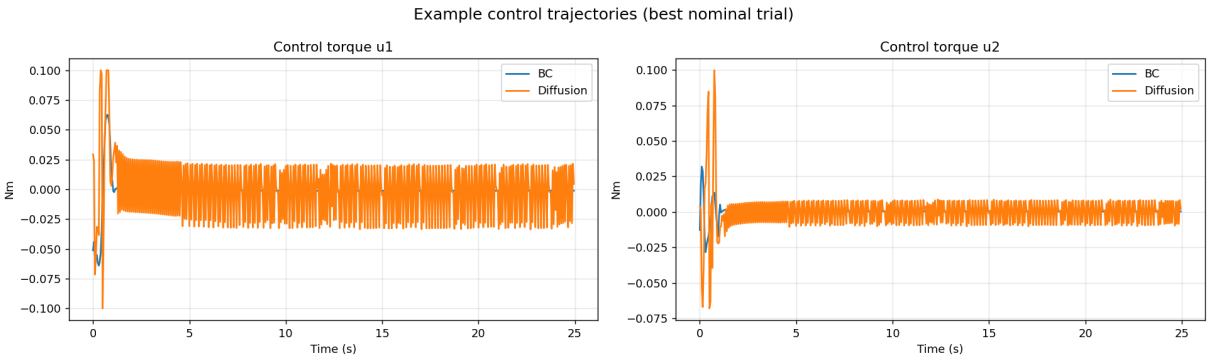


Figure 6: Example control trajectories.

2.5.4 Diffusion design ablations

The two inference choices flagged in Section 2.2 — the transformer backbone and the single-step execution horizon — are not free parameters but the difference between a working controller and

a non-functional one. Figures 7 and 8 make this concrete, sweeping each in isolation over $n = 50$ random-init trials at observation noise $\sigma = 0.02$.

Figure 7 compares noise-prediction backbones at $n_{\text{exec}} = 1$. The trajectory-transformer variants are essentially saturated and insensitive to the number of diffusion steps: $T \in \{5, 10, 20\}$ all land between 96% and 100% (the 78% outlier is a second training run of the T=10 model, indicating the spread is run-to-run variance rather than a systematic gap). The MLP backbone, by contrast, collapses to 2% even at T=100 — discarding the temporal structure of the action chunk is fatal here, which is exactly why the transformer is the default.

Figure 8 sweeps the execution horizon for the T=20 transformer. Re-planning every step ($n_{\text{exec}} = 1$) succeeds 92% of the time, but committing to just two open-loop actions per re-plan drops it to 20%, and four collapses to 0%. The chaotic double-pendulum dynamics diverge from any short open-loop plan, so the closed loop must re-condition on the true state at every control step. This is the empirical basis for the receding-horizon, single-step execution used throughout.

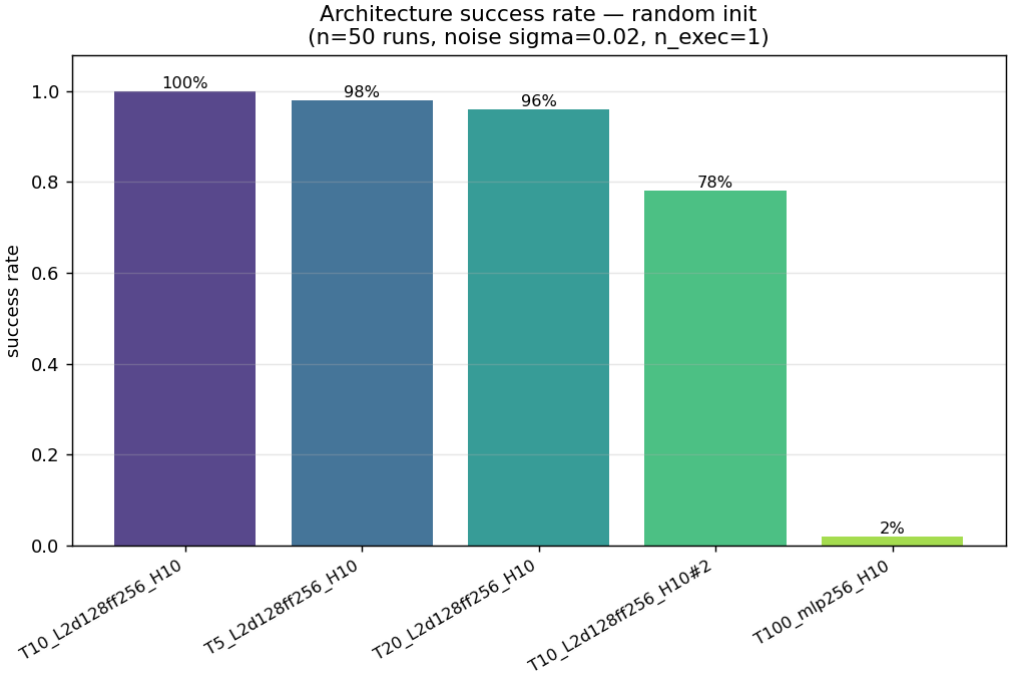


Figure 7: Success rate by noise-prediction backbone (random init, $n = 50$, $\sigma = 0.02$, $n_{\text{exec}} = 1$). Transformer variants saturate across diffusion steps T; the MLP backbone collapses.

Figure 8: Success rate vs. execution steps per re-plan n_{exec} for the T=20 transformer (random init, $n = 50$, $\sigma = 0.02$). Performance is destroyed by committing to more than one open-loop action.

2.5.5 Summary

On this task the two methods are best read as occupying different points on a cost/quality trade-off rather than one strictly dominating the other. The diffusion policy plans better swing-ups — faster and with markedly lower, more consistent trajectory error — which is what one expects from a model that captures the multimodal, temporally correlated action distribution. Behavioural cloning, despite its simplicity, is the more practical controller for *deployment*: it holds the upright with far less jitter, commands far smoother torques, is more robust to observation noise, and is roughly 28× cheaper per control step. For an embedded, real-time swing-up-and-balance controller BC is the pragmatic choice; the diffusion policy's advantages would matter more on tasks with genuinely multimodal expert behaviour, where a unimodal regressor like BC would average incompatible demonstrations together.

3 Robotic Manipulation: Cube Stacking

3.1 Overview & Control Task

This part of the work focuses on the control and manipulation problem in robotic systems using Imitation Learning. Two different approaches are implemented and compared: a baseline Behavior Cloning (BC) method and a Diffusion Policy method. The Franka Emika Panda robot, which is a 7DoF robot, was chosen as the control environment and is simulated in MuJoCo using the robosuite package [2]. The control tasks selected for this work are cube stacking, specifically:

- Stack (2 cubes), where the goal is to place the red cube (Cube A) on top of the green cube (Cube B).
- StackThree (3 cubes), a more complex task as it includes three cubes (Cube A, Cube B, Cube C) that must be placed in the correct order on top of each other.

Robot control is performed in task space using an Operational Space Controller (OSC_POSE). The state space representation and the action space definition are described in detail below:

3.1.1 Action Space

The output of the policy is a 7-dimensional array containing the changes in end-effector movement and gripper state:

- **Translation delta (3 values):** The relative changes of the end-effector position along the x, y, z axes $(\Delta x, \Delta y, \Delta z)$.
- **Rotation delta (3 values):** The relative changes of the end-effector orientation in axis-angle representation.
- **Gripper state (1 value):** A value in the interval $[-1, 1]$ that controls whether the gripper will be in an open or closed position.

3.1.2 State Space

The state space s_t contains the state of the robot and the positions of the objects in the scene. Two configurations are used depending on the dataset:

1. **Stack (2 cubes - state dim = 32):** The state vector consists of:
   - The position and orientation of the two cubes (Cube A and Cube B) in space (14 values: 2 x (3D position + 4D quaternion)).
   - The relative distance and direction from the gripper to Cube A and Cube B (6 values: 2 x 3D vector).
   - The relative distance and direction between the two cubes (3 values: 3D vector).
   - The position and orientation of the end-effector (7 values: 3D position + 4D quaternion).
   - The current angular position of the gripper fingers (2 values).
2. **StackThree (3 cubes - state dim = 48):** For the task with three cubes, the state space vector is extended to include information for Cube C (an additional 16 dimensions). Specifically, the following are added:
   - The position and orientation of Cube C (7 values).
   - The relative distance from the gripper to Cube C (3 values).
   - The relative distances between Cube A-Cube C and Cube B-Cube C (6 values).

3.1.3 Training & Horizon Control Goal

The objective is to learn a policy that generates a sequence of actions over a short horizon:

$$\pi(a_{t:t+H} \mid s_t)$$

where the prediction horizon is defined as H steps. In both models (BC baseline and Diffusion Policy), receding-horizon control is used, where at each time step t the current state s_t is received, a sequence of H actions is generated, the actions are executed, and then the process is repeated by receiving a new state.

3.2 Dataset Overview

The data used to train the models comes from the MimicGen datasets, which are available on Hugging Face (`https://huggingface.co/datasets/amandlek/mimicgen_datasets`). The creation of these datasets is based on MimicGen [1], a system that generates data using initial human demonstrations. MimicGen contains over 48,000 demos across 12 different tasks. Each dataset is saved in hdf5 file format and is compatible with the robomimic library. For each task, a default reset distribution D_0 is defined. Initially, a small number of human demonstrations, typically 10 demos from human experts, are collected on the D_0 distribution. MimicGen then uses this data to generate large datasets across different reset distributions of varying difficulty (D_0, D_1, D_2), for multiple tasks and robots.

The datasets are divided into categories, the two most basic of which are:

- **Source:** The initial data from expert human demonstrations (approximately 10 demonstrations per task).
- **Core:** Data generated automatically via MimicGen alongside the initial source data.

For the implementation of this work, we selected the following four datasets from the core category:

- stack_d0
- stack_d1
- stack_three_d0
- stack_three_d1

Each of these datasets contains exactly 1000 demonstration trajectories. Part of these comes from the initial human expert demonstrations, and the rest were generated automatically using MimicGen, providing the volume of data required to train the Behavior Cloning and Diffusion Policy models.

Table 4 presents a summary of all available observations and shapes per dataset, as obtained from the analysis of the hdf5 files used.

The hdf5 files contain all of the above information (such as positions, velocities, and images); however, for the construction of the state vectors (32D for Stack and 48D for StackThree), only the starred $(*)$ variables were used.

Table 4: Summary of available observations and shapes per dataset (* indicates those used to construct the state vectors).

Observation Key	**stack_d0**	**stack_d1**	**stack_three_d0**	**stack_three_d1**
Total Trajectories	1000	1000	1000	1000
actions*	(102, 7)	(102, 7)	(251, 7)	(255, 7)
agentview_image	(102, 84, 84, 3)	(102, 84, 84, 3)	(251, 84, 84, 3)	(255, 84, 84, 3)
object*	(102, 23)	(102, 23)	(251, 39)	(255, 39)
robot0_eef_pos*	(102, 3)	(102, 3)	(251, 3)	(255, 3)
robot0_eef_quat*	(102, 4)	(102, 4)	(251, 4)	(255, 4)
robot0_eef_vel_ang	(102, 3)	(102, 3)	(251, 3)	(255, 3)
robot0_eef_vel_lin	(102, 3)	(102, 3)	(251, 3)	(255, 3)
robot0_eye_in_hand_image	(102, 84, 84, 3)	(102, 84, 84, 3)	(251, 84, 84, 3)	(255, 84, 84, 3)
robot0_gripper_qpos*	(102, 2)	(102, 2)	(251, 2)	(255, 2)
robot0_gripper_qvel	(102, 2)	(102, 2)	(251, 2)	(255, 2)
robot0_joint_pos	(102, 7)	(102, 7)	(251, 7)	(255, 7)
robot0_joint_pos_cos	(102, 7)	(102, 7)	(251, 7)	(255, 7)
robot0_joint_pos_sin	(102, 7)	(102, 7)	(251, 7)	(255, 7)
robot0_joint_vel	(102, 7)	(102, 7)	(251, 7)	(255, 7)

3.3 Behavior Cloning Model

A Behavior Cloning architecture based on a Multi-Layer Perceptron (MLP) was used as a baseline model. The model is trained to map the state vector directly to an action sequence vector over a given horizon.

3.3.1 Model Architecture

The BC model is based on an architecture of 5 fully connected / linear layers with an increasing and then decreasing number of hidden neurons (from 256 to 512). Layer Normalization, ReLU activation functions, and Dropout with $p = 0.1$ are used between the layers to prevent overfitting. The input to the network is the state vector (dimension 32 or 48), while the final output consists of 56 values corresponding to the actions of the 8 horizon steps for the 7 dimensions of the action space. At the end of the forward pass, the output layer is reshaped from a flat 56-dimensional vector to a tensor with shape ‘(batch_size, 8, 7)‘.

3.3.2 Training & Hyperparameters

An Adam optimizer and the Mean Squared Error (MSE Loss) function between predicted and ground truth expert actions are used for training the model:

$$\text{MSE Loss} = \frac{1}{B \cdot H \cdot A} \sum_{i=1}^{B} \sum_{t=1}^{H} \sum_{a=1}^{A} \left(\hat{a}_{t,a}^{(i)} - a_{t,a}^{(i)} \right)^2$$

where B is the batch size, $H = 8$ is the action horizon, and $A = 7$ is the action dimension.

The basic hyperparameters used for training are summarized in Table 5.

During training, the loss on the validation set is computed after each epoch. The model with the lowest validation loss is exported as the best checkpoint in ‘.pth‘ file format for use in environment testing.

The training progress of the Behavior Cloning model is monitored by evaluating the Mean Squared Error (MSE) loss on both the training and validation sets at the end of each epoch. Figure 9 illustrates the training and validation loss curves over the 50 training epochs.

Table 5: Behavior Cloning training hyperparameters (cube stacking).

Parameter	**Value**
Batch size	64
Learning rate	10^{-3}
Epochs	50
Optimizer	Adam
Loss	MSE
Dataset split	90% train / 10% validation

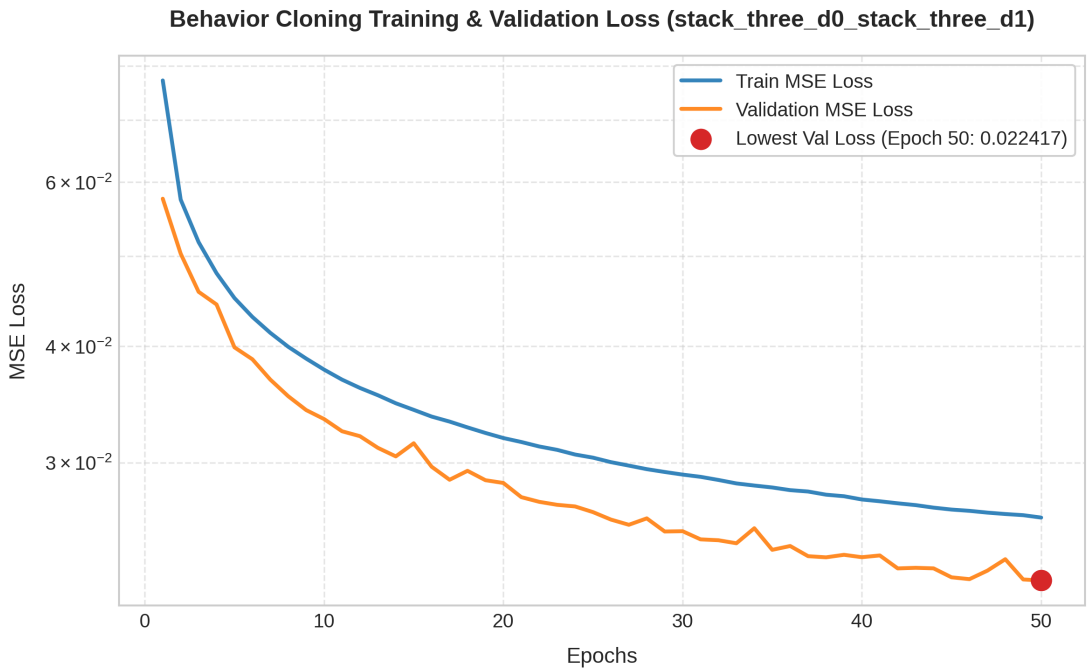


Figure 9: Training and validation Mean Squared Error (MSE) loss curves on a log scale for the Behavior Clone model.

Diffusion Policy Model

The Diffusion Policy model formulates action generation as an iterative denoising process inspired by Denoising Diffusion Probabilistic Models (DDPM). Instead of directly regressing action trajectories as in Behavior Cloning, the model learns to predict the noise that was added to a clean action trajectory and iteratively removes it, starting from pure Gaussian noise and gradually recovering a plausible action sequence conditioned on the current robot state.

1. Model Architecture

The denoiser network is a Transformer encoder (`TransformerDenoiser`) that processes the noisy action trajectory as a sequence of tokens and uses self-attention to model dependencies across the action horizon.

Input Tokenization. The noisy action trajectory of shape (B, H, A_d) is projected into a d-dimensional token sequence through a linear layer, yielding $H = 8$ action tokens. Learnable positional embeddings are added to encode the temporal ordering of the action steps within the horizon.

Timestep and State Conditioning. The diffusion timestep is encoded using sinusoidal positional embeddings and projected through a two-layer MLP with SiLU activations to produce a d-dimensional timestep token. Similarly, the robot state vector (dimension 32 for Stack or 48 for StackThree) is projected through a two-layer MLP with SiLU activations into a state token. Both conditioning tokens are prepended to the action token sequence, forming a combined input sequence of length $H + 2 = 10$ tokens. This token-based conditioning approach allows the transformer's self-attention mechanism to naturally integrate temporal and state information with the action representations.

Transformer Encoder. The token sequence is processed by a stack of 4 standard Transformer encoder layers. Each layer uses Pre-LayerNorm ordering and consists of:

- Multi-head self-attention with $n_h = 4$ attention heads and model dimension $d = 256$
- A feed-forward network with inner dimension $4d = 1024$ and GELU activations
- Dropout with rate $p = 0.1$ applied in both the attention and feed-forward sub-layers
- Residual connections around each sub-layer

Output Projection. After the transformer encoder, only the $H = 8$ action token positions are extracted (the timestep and state conditioning tokens are discarded). A final LayerNorm followed by a linear projection maps the d-dimensional representations back to the action dimension $A_d = 7$, producing the predicted noise tensor with the same shape as the input noisy trajectory.

Noise Schedule. The forward and reverse diffusion processes are managed by a `DiffusionScheduler` that uses a linear variance schedule with β values linearly spaced from 10^{-4} to 0.02 across $K = 100$ timesteps. During training the timestep is normalized by the total number of timesteps ($t_{\text{norm}} = t/K$) before being passed to the sinusoidal embedding. The scheduler supports three reverse sampling strategies: stochastic DDPM, deterministic DDIM, and a probability flow ODE solver with optional Heun (second-order) integration.

2. Training & Hyperparameters

Training Objective. The model is trained with a noise prediction objective. At each training step, a random diffusion timestep k is sampled uniformly from $\{1, \ldots, K-1\}$ and Gaussian noise ϵ is added to the clean (normalized) action trajectory a_0 according to the forward diffusion schedule. The network predicts the noise $\hat{\epsilon}_\theta(a_k, k, s_t)$ and is optimized by minimizing the Mean Squared Error between the predicted and actual noise:

$$\mathcal{L} = \mathbb{E}_{a_0, \epsilon, k}\left[\|\epsilon - \hat{\epsilon}_\theta(a_k, k, s_t)\|^2\right]$$

Training Hyperparameters. The training configuration is summarized in Table 6.

Table 6: Diffusion Policy training hyperparameters (cube stacking).

Parameter	**Value**
Batch size	1024
Learning rate	10^{-4} (cosine annealing, min 10^{-6})
Optimizer	AdamW (weight decay 10^{-4})
Epochs	200
Diffusion timesteps (K)	100
EMA decay	0.995
Mixed-precision	AMP with gradient scaling (GPU)
Dataset split	90% train / 10% validation
Action normalization	element-wise min–max to $[-1, 1]$
Reproducibility	fixed random seeds

Exponential Moving Average (EMA). An Exponential Moving Average of the model weights is maintained throughout training with a decay factor of 0.995. After each optimizer step the EMA weights are updated. The best checkpoint (lowest validation loss) is saved using the EMA weights for inference. EMA smooths out training noise and typically produces better inference results for diffusion models.

Inference Hyperparameters. During deployment in the simulation environment, the inference-time parameters listed in Table 7 are used.

Table 7: Diffusion Policy inference hyperparameters (cube stacking).

Parameter	**Value**
Sampling method	DDPM / DDIM / probability-flow ODE
Inference steps	50
Execution steps	8 (receding horizon)
Action clipping	$[-1, 1]$
Max. episode length	500 steps

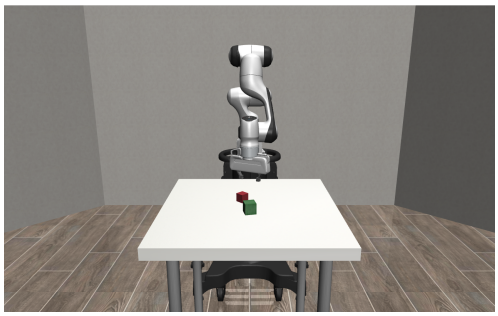
3.4 Testing Environment

The evaluation of the models is performed in the MuJoCo simulation environment using the robosuite library and the mimicgen package. For this purpose, the function `suite.make()` is used to create the environment with the following parameters:

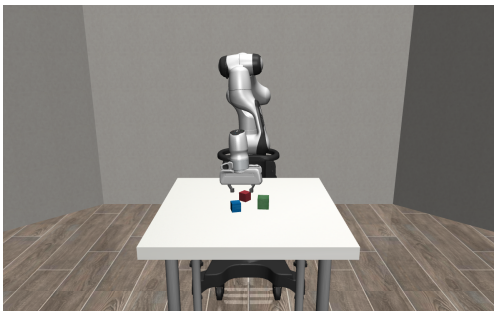
- **env_name ("Stack" or "StackThree"):** Defines the task the robot will execute. In Stack, the goal is to stack 2 cubes, while in StackThree, the goal is to stack 3 cubes.
- **robots ("Panda"):** Selection of the Franka Emika Panda robot (7 DoF).
- **controller_configs:** Selection of the Operational Space Controller (OSC_POSE), which computes the control commands in task space based on end-effector position and orientation.
- **has_renderer (True) & has_offscreen_renderer (False):** Enables the simulation window for visual monitoring of the robot and disables offscreen rendering to save resources.
- **use_camera_obs (False):** Disables camera observations since this data was not used during training.

- **control_freq (20):** Control frequency set to 20 Hz, which corresponds to a cycle time of 50ms per step and matches the sampling frequency of the datasets.
- **horizon (500):** The maximum rollout steps per episode, limiting the simulation time to 25 seconds (500 steps $\times$ 0.05 s/step).

To extract the results, 50 independent evaluation episodes were executed for each model and environment. At each time step, the predicted actions are executed and it is checked whether cube stacking has been achieved (success state). The simulation setups for both tasks are shown in Figure 10.



(a) The two-cube stacking environment (Stack).



(b) The three-cube stacking environment (StackThree).

Figure 10: The cube-stacking simulation environments used for evaluation.

3.5 Evaluation Results

To evaluate the quality and efficiency of the control trajectories generated by the models, we define three performance metrics that are computed at each planning step k of a rollout episode. Let the model's action sequence prediction at planning step k be:

$$A^{(k)} = \left[a_1^{(k)}, a_2^{(k)}, \ldots, a_{H_p}^{(k)}\right] \in \mathbb{R}^{H_p \times A_d}$$

where $H_p = 8$ is the prediction horizon, $A_d = 7$ is the action space dimension, and $a_{t,j}^{(k)}$ represents the command at time step t for action dimension j. The metrics are formulated as follows:

1. **Trajectory Jitter (J):** Measures the smoothness of the predicted action trajectory by computing the mean squared difference between consecutive action steps:

$$J^{(k)} = \frac{1}{(H_p - 1)A_d} \sum_{t=1}^{H_p-1} \sum_{j=1}^{A_d} \left(a_{t+1,j}^{(k)} - a_{t,j}^{(k)}\right)^2$$

   For an episode consisting of N planning steps, the episode jitter is the average over all planning steps: $J_{\text{episode}} = \frac{1}{N} \sum_{k=1}^{N} J^{(k)}$.

2. **Path Effort (E):** Measures the total magnitude of spatial end-effector translation commands (the first 3 dimensions representing relative translation x, y, z):

$$E^{(k)} = \sum_{t=1}^{H_p} \sum_{j=1}^{3} \left|a_{t,j}^{(k)}\right|$$

   The total episode effort is the sum of step efforts: $E_{\text{episode}} = \sum_{k=1}^{N} E^{(k)}$.

3. **Joint Cost (C):** Measures the total command magnitude across all 7 action dimensions (representing both end-effector control and gripper command magnitude):

$$C^{(k)} = \sum_{t=1}^{H_p} \sum_{j=1}^{A_d} \left|a_{t,j}^{(k)}\right|$$

   The total episode joint cost is the sum of step joint costs: $C_{\text{episode}} = \sum_{k=1}^{N} C^{(k)}$.

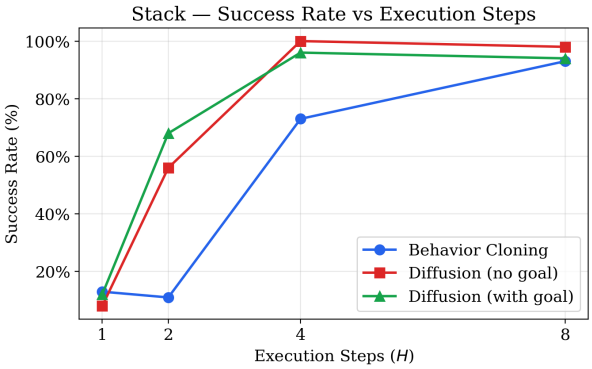
Following, we present the evaluation results of the trained models in the simulation environment. Table 8 summarizes the performance metrics of both the Behavior Cloning (BC) baseline and the Diffusion Policy model across different execution horizons (`EXECUTE_STEPS` $H \in \{1, 2, 4, 8\}$) for the Stack (2 cubes) and StackThree (3 cubes) tasks.

Both models benefit from longer execution horizons, with $H = 8$ consistently yielding high success rates and the lowest effort scores. However, the Diffusion Policy demonstrates a clear advantage in task success: on Stack it reaches 100.0% at $H = 4$ and 98.0% at $H = 8$, whereas BC peaks at only 93.0% (at $H = 8$). On the more challenging StackThree task, the diffusion model attains 70.0% at both $H = 4$ and $H = 8$ compared to BC's 56.0%, and already reaches 36.0% at $H = 2$ where BC succeeds only 1.0% of the time. This robustness to replanning frequency is a practical advantage, as the policy can adapt more quickly to environmental changes without sacrificing reliability. In terms of trajectory quality, BC produces smoother action sequences with lower jitter values across all configurations (e.g. 0.00676 vs. 0.02154 at $H = 8$ on Stack), which is expected given its deterministic regression nature compared to the stochastic denoising process. Despite this, the diffusion model's higher jitter does not impair task completion, as confirmed by its superior success rates. Path effort and joint cost are comparable between the two models at $H = 8$, with the diffusion model achieving slightly lower path effort on Stack (96.91 vs.

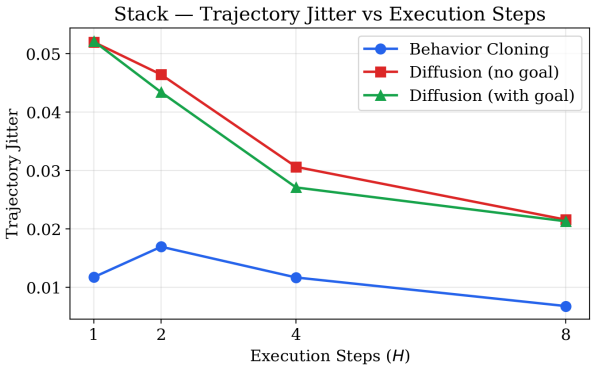
Table 8: Empirical evaluation results of the Behavior Cloning (BC) and Diffusion Policy models across different execution steps (H) on the Stack and StackThree environments (values reported as Mean / Median).

Task	**Model**	H	**Success**	**Jitter**	**Path Effort**	**Joint Cost**
Stack	BC	1	13.0%	0.01170 / 0.01029	2073.22 / 2031.59	5515.72 / 5547.95
		2	11.0%	0.01691 / 0.01577	934.09 / 908.48	2714.83 / 2768.91
		4	73.0%	0.01165 / 0.01099	269.30 / 171.67	765.77 / 484.45
		8	**93.0%**	**0.00676 / 0.00623**	**103.05 / 82.74**	**268.03 / 218.65**
	Diffusion	1	8.0%	0.05197 / 0.05309	1713.39 / 1393.61	7210.27 / 6949.14
		2	56.0%	0.04638 / 0.04923	614.44 / 516.66	2567.56 / 2626.39
		4	**100.0%**	0.03059 / 0.02780	202.64 / 176.12	650.03 / 569.80
		8	98.0%	**0.02154 / 0.02063**	**96.91 / 86.39**	**292.41 / 263.00**
StackThree	BC	1	2.0%	0.00468 / 0.00438	2131.12 / 2143.83	6189.94 / 6207.12
		2	1.0%	0.00514 / 0.00529	1052.04 / 998.69	3104.60 / 3068.19
		4	25.0%	0.00539 / 0.00512	462.80 / 438.15	1389.38 / 1398.30
		8	**56.0%**	**0.00366 / 0.00374**	**232.03 / 202.94**	**652.26 / 579.43**
	Diffusion	1	16.0%	0.02755 / 0.03086	2155.16 / 2126.18	7617.41 / 7847.57
		2	36.0%	0.02332 / 0.01884	1124.72 / 1000.23	3645.99 / 3823.49
		4	**70.0%**	0.01904 / 0.01586	470.53 / 447.22	1520.41 / 1388.01
		8	**70.0%**	**0.01417 / 0.01391**	**238.20 / 217.04**	**740.25 / 644.47**

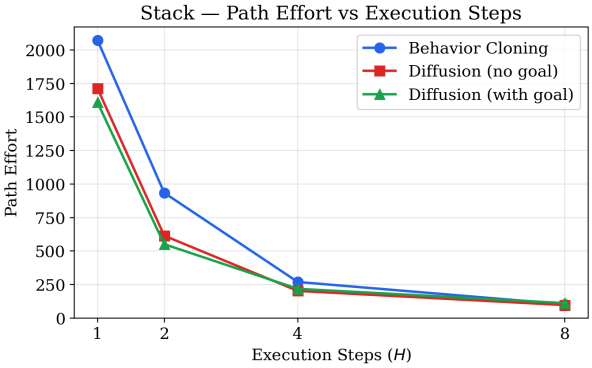
103.05) while showing marginally higher joint cost on StackThree (740.25 vs. 652.26), likely due to additional corrective movements needed to complete the more complex three-cube stacking sequence. At low execution horizons ($H = 1$), both models struggle: BC suffers from compounding errors and inability to properly trigger gripper state changes, while the diffusion model produces excessively noisy trajectories with high jitter and effort, indicating that single-step execution disrupts the temporal coherence of the generated action plans. These trends are visualized in Figure 11 for the Stack task and Figure 12 for StackThree, which plot each metric against the execution horizon H.



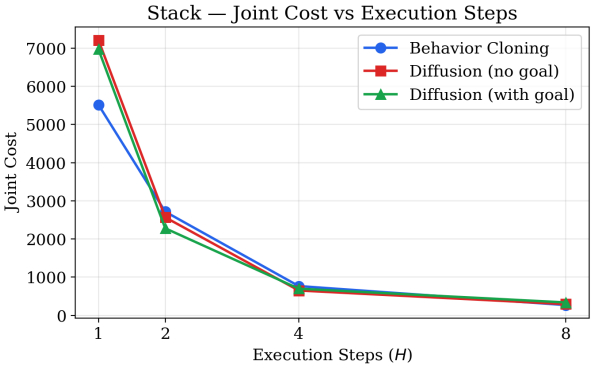
(a) Success rate.



(b) Trajectory jitter.

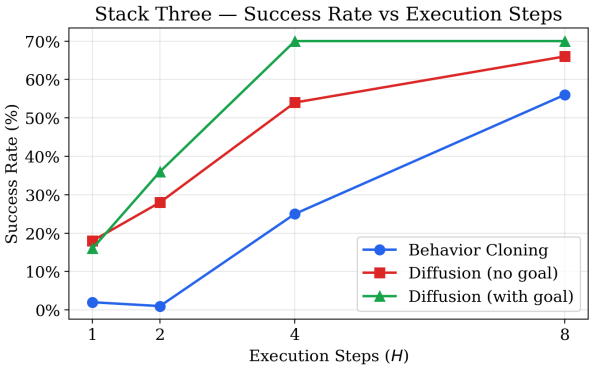


(c) Path effort.

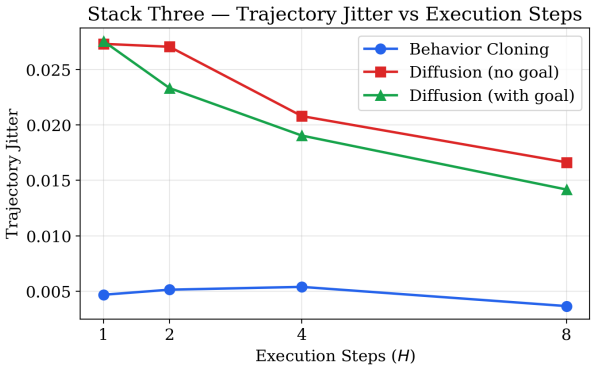


(d) Joint cost.

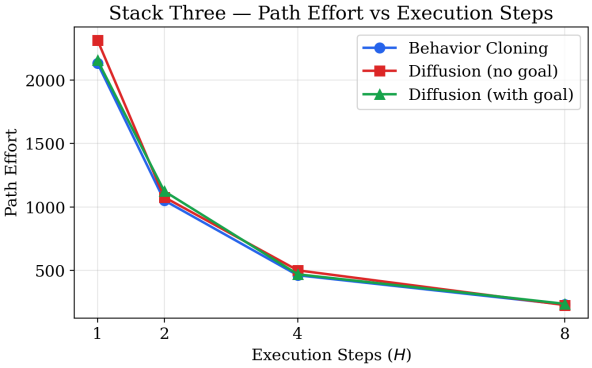
Figure 11: Behavior Cloning vs. Diffusion Policy on the Stack task across execution horizons $H \in \{1, 2, 4, 8\}$, for each of the four evaluation metrics.



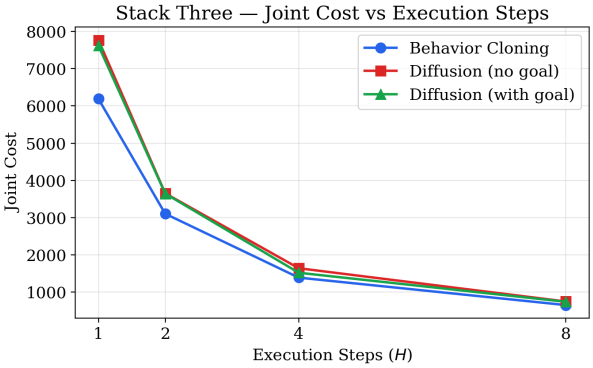
(a) Success rate.



(b) Trajectory jitter.



(c) Path effort.



(d) Joint cost.

Figure 12: Behavior Cloning vs. Diffusion Policy on the StackThree task across execution horizons $H \in \{1, 2, 4, 8\}$, for each of the four evaluation metrics.

4 Conclusion

We compared two imitation-learning methods — behavioural cloning (BC) and a diffusion policy — on two tasks: the double-pendulum swing-up and cube stacking (Stack and StackThree). For the pendulum the expert data came from a trajectory optimizer tracked by a TVLQR controller and run in the MuJoCo simulator; for stacking it came from the MimicGen demonstrations of a 7-DoF Franka Panda robot. Since both methods were trained on the same data for each task, the comparison reflects the choice of method rather than the data.

On the **double pendulum** both methods swing the pendulum up and hold it, so neither one fails the task. On this task, though, BC is the better controller and the diffusion policy is the weaker one. The diffusion policy is sensitive to how often it replans: it only works if it computes a new action every step, and committing to even two actions in a row makes it drift and fall (Figure 8). Even when it succeeds it holds the top less cleanly, with about 8× rougher torque and much more jitter than BC, it is more affected by measurement noise (90% vs. BC's 100% over random starts), and it is about 28× more expensive per step. It does swing up somewhat faster and with lower tracking error (IAE 8.98 vs. 14.6 rad·s), but that does not outweigh the sensitivity and the jitter. For this task BC is the more sensible choice.

On **cube stacking** it is the other way around: here BC is the weaker method and the diffusion policy is the better one. The diffusion policy stacks the cubes more reliably (98% vs. 93% on Stack, 74% vs. 56% on the harder StackThree), and it keeps working when replanning more often, where BC drops off sharply — for example 62% vs. BC's 1% at $H = 2$ on StackThree. BC's action sequences are smoother, but on this task that does not help, because it completes the stack less often. The demonstrations contain several valid ways to finish the task, and BC tends to blur them into one average motion while the diffusion policy can keep them apart.

The two tasks also need opposite replanning rates, and each method happens to suit one of them. The pendulum must replan every step, because its fast, unstable motion drifts away from any longer plan; the stacking tasks work best with the long $H = 8$ horizon and fail at $H = 1$, where replanning every step breaks up the motion. So neither method is best everywhere: BC handles the small, fast balance task well but struggles with stacking, and the diffusion policy handles stacking well but is fragile on the pendulum. In short, BC is the simpler and cheaper choice for smooth control of small, balance-type problems, and the diffusion policy is the better choice for larger tasks such as manipulation, where the same job can be done in several different ways.

References

[1] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, "MimicGen: A Data Generation System for Scalable Robot Learning using Human Demonstrations," in *7th Annual Conference on Robot Learning*, 2023.

[2] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu, "robosuite: A Modular Simulation Framework and Benchmark for Robot Learning," in *arXiv preprint arXiv:2009.12293*, 2020.

[3] S. J. D. Prince, *Understanding Deep Learning*. Cambridge, MA: MIT Press, 2023. [Online]. Available: `https://udlbook.github.io/udlbook/`